Shiksha Mandal's
Bajaj Institute of Technology, Wardha
(Recognized under section 2(f) of UGC Act 1956)
(Affiliated to DBATU, Lonere, Raigad, Maharashtra)

Academic Session: 2026-27
BITACAD/OS/COMP/2026-27/Training/VII/1
Date: 02/09/2026

Department of Computer Engineering

Technical Interview Question & Model Answer Bank

Academic Year	**2026-2027**	**Class**	**4th year**
Subject/Expert Area	**Java Programming**		
Expert Name	**Mr. S D jain, Mr. A. D. Saraf**		

Question/Model Answer Format (File Name : <Subject Name>.docx

Sr. No.	Question	Model Answer	Practical Example

1	**Question 1: Employee Salary Processing**
	Scenario
	A company maintains employee information containing employee ID, name, department, and salary. The HR department wants to display employees earning more than Rs. 50,000 in descending order of salary.
	Topics Covered
	• Classes and Objects
	• Constructors
	• ArrayList
	• Lambda Expressions
	• Java Streams
	• filter()
	• sorted()
	• Comparator
	Starter Code
	`import java.util.*;`
	`class Employee {`
	`  int id;`
	`  String name;`
	`  String department;`
	`  double salary;`
	`  Employee(int id, String name, String department, double salary) {`
	`  this.id = id;`
	`  this.name = name;`
	`  this.department = department;`
	`  this.salary = salary;`
	`  }`
	`}`
	`public class EmployeeSalaryDemo {`
	`  public static void main(String[] args) {`
	`  List<Employee> employees = new ArrayList<>();`
	`  employees.add(new Employee(101, "Amit", "IT", 65000));`
	`  employees.add(new Employee(102, "Neha", "HR", 45000));`
	`  employees.add(new Employee(103, "Rahul", "IT", 75000));`
	`  employees.add(new Employee(104, "Priya", "Finance", 55000));`
	`  // TODO 1: Filter employees having salary > 50000`
	`  // TODO 2: Sort them in descending order of salary`
	`  // TODO 3: Display employee name and`
		Step-by-Step Logic
		Step 1: Create an Employee class to represent each employee as one object.
		Step 2: Store Employee objects in an ArrayList so that the number of employees can grow dynamically.
		Step 3: Convert the list to a stream with stream().
		Step 4: Use filter(e -> e.salary > 50000) to retain only employees satisfying the salary condition.
		Step 5: Use sorted(...) with a Comparator that compares salary and reverses the order.
		Step 6: Use forEach(...) to print the selected employees.
		Complete Java Solution
		`import java.util.*;`
		`class Employee {`
		`  int id;`
		`  String name;`
		`  String department;`
		`  double salary;`
		`  Employee(int id, String name, String department, double salary) {`
		`  this.id = id;`
		`  this.name = name;`
		`  this.department = department;`
		`  this.salary = salary;`
		`  }`
		`}`
		`public class EmployeeSalaryDemo {`
		`  public static void main(String[] args) {`
		`  List<Employee> employees = new ArrayList<>();`
		`  employees.add(new Employee(101, "Amit", "IT", 65000));`
		`  employees.add(new Employee(102, "Neha", "HR", 45000));`
		`  employees.add(new Employee(103, "Rahul", "IT", 75000));`
		`  employees.add(new Employee(104, "Priya", "Finance", 55000));`
		`  employees.stream()`
		`  .filter(e -> e.salary > 50000)`
		`  .sorted(Comparator.comparingDouble((Employee e) -> e.salary).reversed())`
		`  .forEach(e -> System.out.println(e.name + " - " + e.salary));`
		`  }`
		`}`
		Expected Output
		Rahul - 75000.0	

```
salary
    }
}
```

Amit - 65000.0
Priya - 55000.0

Important Interview Points

• filter() does not modify the original list; it builds a stream pipeline.

• Time complexity is dominated by sorting: O(k log k), where k is the number of filtered employees.

2

Question 2: Remove Duplicate Customers

Scenario

An online shopping application contains duplicate customer records. Two customers are considered duplicates if their customerId is the same. Remove duplicate records while maintaining the original insertion order.

Topics Covered

• List
• Set
• LinkedHashSet
• Object Comparison
• equals()
• hashCode()
• Collections

Starter Code

```
import java.util.*;

class Customer {
    int customerId;
    String name;

    Customer(int customerId, String name) {
        this.customerId = customerId;
        this.name = name;
    }

    // TODO: Override equals()

    // TODO: Override hashCode()
}

public class CustomerDemo {
    public static void main(String[] args) {
        List<Customer> customers = new
ArrayList<>();

        customers.add(new Customer(101, "Amit"));
        customers.add(new Customer(102,
"Priya"));
        customers.add(new Customer(101, "Amit"));
        customers.add(new Customer(103,
"Rahul"));

        // TODO: Remove duplicate customers
        // while preserving insertion order.
    }
}
```

Step-by-Step Logic

Step 1: Decide what makes two Customer objects logically equal. Here, customerId is the identity field.

Step 2: Override equals() so two customers with the same customerId compare as equal.

Step 3: Override hashCode() using the same field used by equals(). This is required for hash-based collections.

Step 4: Create a LinkedHashSet from the original list. A Set removes duplicates, while LinkedHashSet also preserves insertion order.

Step 5: If a List is required afterward, create a new ArrayList from the LinkedHashSet.

Complete Java Solution

```
import java.util.*;

class Customer {
    int customerId;
    String name;

    Customer(int customerId, String name) {
        this.customerId = customerId;
        this.name = name;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Customer)) return false;
        Customer other = (Customer) obj;
        return customerId == other.customerId;
    }

    @Override
    public int hashCode() {
        return Integer.hashCode(customerId);
    }

    @Override
    public String toString() {
        return customerId + " - " + name;
    }
}

public class CustomerDemo {
    public static void main(String[] args) {
        List<Customer> customers = new ArrayList<>();
        customers.add(new Customer(101, "Amit"));
        customers.add(new Customer(102, "Priya"));
        customers.add(new Customer(101, "Amit
Duplicate"));
        customers.add(new Customer(103, "Rahul"));

        Set<Customer> uniqueSet = new
LinkedHashSet<>(customers);
        List<Customer> uniqueCustomers = new
ArrayList<>(uniqueSet);

        uniqueCustomers.forEach(System.out::println);
    }
}
```

Expected Output

101 - Amit
102 - Priya
103 - Rahul

Important Interview Points

• Always override hashCode() when equals() is overridden.

• HashSet removes duplicates but does not promise insertion order; LinkedHashSet does.

3

Question 3: Student Marks Analysis

Scenario

A college stores marks of students in five subjects. The system should calculate each student's average marks, identify students scoring above 75%, and find the class topper.

Topics Covered

- Arrays
- Classes
- Loops
- Methods
- Aggregation
- Conditional Statements

Starter Code

```
class Student {
    String name;
    int[] marks;

    Student(String name, int[] marks) {
        this.name = name;
        this.marks = marks;
    }

    double calculateAverage() {
        // TODO: Calculate total marks
        // TODO: Return average
        return 0;
    }
}

public class StudentResult {
    public static void main(String[] args) {
        Student[] students = {
            new Student("Amit", new int[]{78, 85, 72, 90, 80}),
            new Student("Priya", new int[]{92, 88, 95, 91, 89}),
            new Student("Rahul", new int[]{65, 70, 68, 75, 72})
        };

        // TODO: Display average of every student
        // TODO: Display students with average > 75
        // TODO: Find topper
    }
}
```

Step-by-Step Logic

Step 1: Store each student's marks in an int array.

Step 2: Create calculateAverage() so the averaging logic belongs to the Student class.

Step 3: Loop through the marks, add them, and divide by the number of subjects.

Step 4: In main(), calculate and print every student's average.

Step 5: Track the highest average seen so far and keep a reference to that student.

Step 6: Use an if condition to display students whose average is greater than 75.

Complete Java Solution

```
class Student {
    String name;
    int[] marks;

    Student(String name, int[] marks) {
        this.name = name;
        this.marks = marks;
    }

    double calculateAverage() {
        int total = 0;
        for (int mark : marks) {
            total += mark;
        }
        return (double) total / marks.length;
    }
}

public class StudentResult {
    public static void main(String[] args) {
        Student[] students = {
            new Student("Amit", new int[]{78, 85, 72, 90, 80}),
            new Student("Priya", new int[]{92, 88, 95, 91, 89}),
            new Student("Rahul", new int[]{65, 70, 68, 75, 72})
        };

        Student topper = null;
        double highestAverage = -1;

        for (Student s : students) {
            double average = s.calculateAverage();
            System.out.printf("%s average = %.2f%%%n", s.name, average);

            if (average > 75) {
                System.out.println("  Above 75%");
            }

            if (average > highestAverage) {
                highestAverage = average;
                topper = s;
            }
        }

        System.out.printf("Topper: %s (%.2f%%)%n", topper.name, highestAverage);
    }
}
```

Expected Output

Amit average = 81.00%
  Above 75%
Priya average = 91.00%
  Above 75%
Rahul average = 70.00%
Topper: Priya (91.00%)

Important Interview Points

- Casting total to double prevents integer division.
- The topper can be found in the same pass, so no sorting is required.

4

Question 4: Word Frequency in Customer Reviews

Step-by-Step Logic

Step 1: Normalize the sentence to lower case so Java and java are

Scenario

An e-commerce application receives the review: "Java is powerful and Java is easy to learn". Count how many times each word occurs.

Topics Covered

• Strings
• split()
• Arrays
• HashMap
• Map Operations
• Frequency Counting

Starter Code

```
import java.util.*;

public class WordFrequency {
    public static void main(String[] args) {
        String review = "Java is powerful and
Java is easy to learn";
        String[] words = review.split(" ");
        Map<String, Integer> frequency = new
HashMap<>();

        for (String word : words) {
            // TODO: Check whether word exists
            // TODO: Increase frequency
        }

        // TODO: Display words and frequencies
    }
}
```

treated as the same word.

Step 2: Split the sentence into words using whitespace.

Step 3: Use a HashMap<String,Integer> where the key is the word and the value is the count.

Step 4: For every word, use merge(word, 1, Integer::sum) to insert 1 when absent or increment when present.

Step 5: Traverse the map and print each frequency.

Complete Java Solution

```
import java.util.*;

public class WordFrequency {
    public static void main(String[] args) {
        String review = "Java is powerful and Java is easy
to learn";
        String[] words =
review.toLowerCase().split("\\s+");

        Map<String, Integer> frequency = new
LinkedHashMap<>();

        for (String word : words) {
            frequency.merge(word, 1, Integer::sum);
        }

        for (Map.Entry<String, Integer> entry :
frequency.entrySet()) {
            System.out.println(entry.getKey() + " = " +
entry.getValue());
        }
    }
}
```

Expected Output

java = 2
is = 2
powerful = 1
and = 1
easy = 1
to = 1
learn = 1

Important Interview Points

• HashMap is a natural choice for frequency counting because lookup/update is O(1) on average.
• LinkedHashMap is used here only so the displayed order follows first occurrence.

5

Question 5: Bank Withdrawal with Custom Exception

Scenario

A bank should reject a withdrawal when the withdrawal amount exceeds the account balance. Create and use a custom InsufficientBalanceException.

Topics Covered

• Exception Handling
• Custom Exceptions
• throw
• throws
• try-catch
• Classes

Starter Code

```
class InsufficientBalanceException extends
Exception {
    public InsufficientBalanceException(String
message) {
        super(message);
    }
}

class BankAccount {
    double balance = 10000;

    public void withdraw(double amount)
```

Step-by-Step Logic

Step 1: Create InsufficientBalanceException by extending Exception, making it a checked exception.

Step 2: Declare withdraw() with throws InsufficientBalanceException.

Step 3: Validate that the amount is positive and that enough balance is available.

Step 4: Use throw new InsufficientBalanceException(...) when the balance is insufficient.

Step 5: Call withdraw() inside a try block and handle the custom exception in catch.

Complete Java Solution

```
class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) {
        super(message);
    }
}

class BankAccount {
    private double balance = 10000;

    public void withdraw(double amount) throws
InsufficientBalanceException {
        if (amount <= 0) {
            throw new IllegalArgumentException("Withdrawal
amount must be positive.");
        }

        if (amount > balance) {
```

```
            throws InsufficientBalanceException {
        // TODO: Check if amount > balance
        // TODO: Throw exception if necessary
        // TODO: Deduct amount
    }
}

public class BankDemo {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

        try {
            account.withdraw(12000);
        } catch (InsufficientBalanceException e)
{
            // TODO: Display appropriate message
        }
    }
}
```

```
            throw new InsufficientBalanceException(
                    "Insufficient balance. Available
balance = " + balance);
        }

        balance -= amount;
        System.out.println("Withdrawal successful.
Remaining balance = " + balance);
    }
}

public class BankDemo {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

        try {
            account.withdraw(12000);
        } catch (InsufficientBalanceException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Expected Output

Insufficient balance. Available balance = 10000.0

Important Interview Points

• Use a custom exception when the failure has business meaning that callers may want to handle separately.

• IllegalArgumentException is appropriate for an invalid negative/zero input because that is a programming/input contract error.

6

Question 6: Product Sorting

Scenario

An online shopping system stores productId, productName, price, and rating. Sort products first by rating in descending order and, when ratings are equal, by price in ascending order.

Topics Covered

• Collections

• Comparator

• Lambda Expressions

• Multiple Sorting Criteria

• Objects

Starter Code

```
import java.util.*;

class Product {
    int id;
    String name;
    double price;
    double rating;

    Product(int id, String name, double price,
double rating) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.rating = rating;
    }
}

public class ProductSorting {
    public static void main(String[] args) {
        List<Product> products = new
ArrayList<>();

        products.add(new Product(1, "Laptop",
60000, 4.5));
        products.add(new Product(2, "Mobile",
25000, 4.7));
        products.add(new Product(3, "Tablet",
30000, 4.5));

        // TODO: Create Comparator
        // TODO: Sort rating descending
        // TODO: If ratings equal, sort by price
ascending
        // TODO: Display sorted products
```

Step-by-Step Logic

Step 1: Create Product objects and place them in a List.

Step 2: Build the first comparator using comparingDouble(Product -> rating).

Step 3: Call reversed() immediately on the rating comparator to obtain descending rating order.

Step 4: Chain thenComparingDouble(Product -> price) so equal ratings are ordered by lower price first.

Step 5: Call products.sort(comparator) and display the result.

Complete Java Solution

```
import java.util.*;

class Product {
    int id;
    String name;
    double price;
    double rating;

    Product(int id, String name, double price, double
rating) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.rating = rating;
    }
}

public class ProductSorting {
    public static void main(String[] args) {
        List<Product> products = new ArrayList<>();
        products.add(new Product(1, "Laptop", 60000,
4.5));
        products.add(new Product(2, "Mobile", 25000,
4.7));
        products.add(new Product(3, "Tablet", 30000,
4.5));
        products.add(new Product(4, "Monitor", 20000,
4.5));

        Comparator<Product> comparator =
                Comparator.comparingDouble((Product p) ->
p.rating)
                        .reversed()
                        .thenComparingDouble(p ->
p.price);

        products.sort(comparator);
```

	`    }`
	`}`
		`        for (Product p : products) {`
		`            System.out.println(p.name + " rating=" + p.rating + " price=" + p.price);`
		`        }`
		`    }`
		`}`
		Expected Output
		Mobile rating=4.7 price=25000.0
		Monitor rating=4.5 price=20000.0
		Tablet rating=4.5 price=30000.0
		Laptop rating=4.5 price=60000.0
		Important Interview Points
		• Be careful with reversed(): reversing the entire chained comparator would also reverse the price order.
		• Comparator chaining is preferable to nested if statements because it is clearer and reusable.	

7	**Question 7: Find the Second Highest Salary**
	Scenario
	An HR application stores employee salaries, and some employees may have the same salary. Find the second highest distinct salary without manually sorting the complete list.
	Topics Covered
	• Collections
	• Set
	• Streams
	• Distinct Values
	• Sorting
	• Problem Solving
	Starter Code
	`import java.util.*;`
	
	`public class SecondHighestSalary {`
	`    public static void main(String[] args) {`
	`        List<Integer> salaries = Arrays.asList(`
	`            50000, 70000, 60000, 70000, 80000`
	`        );`
	
	`        // TODO: Remove duplicate salaries`
	`        // TODO: Find highest salary`
	`        // TODO: Find second highest distinct salary`
	`    }`
	`}`
	Student Task
	• Expected result for the sample data: 70000.
	• Additional challenge: solve it without sorting the complete list.
		Step-by-Step Logic
		Step 1: Keep two variables: highest and secondHighest.
		Step 2: For each salary, first check whether it is already equal to highest; if so, ignore the duplicate.
		Step 3: If salary is greater than highest, move the old highest into secondHighest and store the new value as highest.
		Step 4: Otherwise, if salary is less than highest but greater than secondHighest, update secondHighest.
		Step 5: After one pass, secondHighest contains the second highest distinct salary.
		Complete Java Solution
		`import java.util.*;`
		
		`public class SecondHighestSalary {`
		`    public static void main(String[] args) {`
		`        List<Integer> salaries = Arrays.asList(50000, 70000, 60000, 70000, 80000);`
		
		`        Integer highest = null;`
		`        Integer secondHighest = null;`
		
		`        for (int salary : salaries) {`
		`            if (highest == null || salary > highest) {`
		`                secondHighest = highest;`
		`                highest = salary;`
		`            } else if (salary != highest &&`
		`                    (secondHighest == null || salary > secondHighest)) {`
		`                secondHighest = salary;`
		`            }`
		`        }`
		
		`        if (secondHighest != null) {`
		`            System.out.println("Second highest distinct salary = " + secondHighest);`
		`        } else {`
		`            System.out.println("A second distinct salary does not exist.");`
		`        }`
		`    }`
		`}`
		Expected Output
		Second highest distinct salary = 70000
		Important Interview Points
		• This solution runs in O(n) time and O(1) extra space.
		• The word distinct is important: duplicate occurrences of the highest salary must not become the second highest.	

8	**Question 8: Inventory Management System** **Scenario** A shop maintains product quantities using the product name as the key and available quantity as the value. Implement common inventory operations. **Topics Covered** • HashMap	**Step-by-Step Logic** Step 1: Use a Map<String,Integer> because every product name maps to one current quantity. Step 2: Use put() to add a new product or replace a known quantity. Step 3: Use computeIfPresent() or put() to update stock. Step 4: Use remove() to delete a product. Step 5: Traverse entrySet() and print entries whose quantity is less	

• CRUD Operations
• Loops
• Conditional Statements
• Map Traversal

Starter Code

```
import java.util.*;

public class InventorySystem {
    public static void main(String[] args) {
        Map<String, Integer> inventory = new
HashMap<>();

        inventory.put("Laptop", 10);
        inventory.put("Mouse", 25);
        inventory.put("Keyboard", 5);

        // TODO 1: Add a new product
        // TODO 2: Update Laptop quantity
        // TODO 3: Remove one product
        // TODO 4: Display products having stock
< 10
    }
}
```

than 10.

Complete Java Solution

```
import java.util.*;

public class InventorySystem {
    public static void main(String[] args) {
        Map<String, Integer> inventory = new HashMap<>();

        inventory.put("Laptop", 10);
        inventory.put("Mouse", 25);
        inventory.put("Keyboard", 5);

        // Add a new product.
        inventory.put("Webcam", 7);

        // Add 5 units to Laptop stock.
        inventory.computeIfPresent("Laptop", (product,
qty) -> qty + 5);

        // Remove a product.
        inventory.remove("Mouse");

        System.out.println("Low-stock products:");
        for (Map.Entry<String, Integer> entry :
inventory.entrySet()) {
            if (entry.getValue() < 10) {
                System.out.println(entry.getKey() + " = "
+ entry.getValue());
            }
        }
    }
}
```

Expected Output

Low-stock products:
Keyboard = 5
Webcam = 7

Important Interview Points

• A Map avoids scanning a List when the product key is already known.

• HashMap does not preserve display order. Use LinkedHashMap if insertion order matters.

9

Question 9: Immutable User Class

Scenario

A security-sensitive application stores userId, username, and registrationDate. Once a User object is created, its state should not be modifiable.

Topics Covered

• Immutability
• final
• Encapsulation
• Defensive Copying
• Constructors
• Getter Methods

Starter Code

```
import java.util.Date;

final class User {
    private final int userId;
    private final String username;
    private final Date registrationDate;

    User(int userId, String username, Date
registrationDate) {
        this.userId = userId;
        this.username = username;

        // TODO: Use defensive copying
        this.registrationDate = null;
    }

    // TODO: Create getters

    // Important:
    // Do not return mutable Date directly.
```

Step-by-Step Logic

Step 1: Declare the class final so it cannot be subclassed and made mutable through overriding.

Step 2: Declare all fields private and final.

Step 3: Initialize all fields in the constructor and do not provide setters.

Step 4: Because Date is mutable, copy it in the constructor instead of storing the caller's Date reference.

Step 5: Return another copy from the getter so callers cannot mutate the internal Date object.

Complete Java Solution

```
import java.util.Date;

final class User {
    private final int userId;
    private final String username;
    private final Date registrationDate;

    User(int userId, String username, Date
registrationDate) {
        this.userId = userId;
        this.username = username;
        this.registrationDate = new
Date(registrationDate.getTime());
    }

    public int getUserId() {
        return userId;
    }

    public String getUsername() {
        return username;
    }

    public Date getRegistrationDate() {
        return new Date(registrationDate.getTime());
```

```
}
```

Student Task

• Explain why returning registrationDate directly from a getter can break immutability.

```
    }
}

public class ImmutableUserDemo {
    public static void main(String[] args) {
        Date date = new Date();
        User user = new User(101, "Amit", date);

        // Attempt to change the original Date after
construction.
        date.setTime(0);

        System.out.println(user.getUserId());
        System.out.println(user.getUsername());
        System.out.println(user.getRegistrationDate());
    }
}
```

Expected Output

101
Amit
<the original registration date/time, not 1-Jan-1970>

Important Interview Points

• String is already immutable, so it can safely be stored directly.

• If the class used LocalDate/Instant instead of Date, defensive copying would not be needed because those types are immutable.

10

Question 10: Railway Seat Booking

Scenario

There is only one train seat available. Two users try to book it simultaneously. Prevent double booking.

Topics Covered

• Multithreading
• Threads
• Race Conditions
• Synchronization
• Shared Resources

Starter Code

```
class TicketBooking {
    int availableSeats = 1;

    public void bookSeat(String user) {
        // TODO: Make this operation thread-safe
        if (availableSeats > 0) {
            System.out.println(user + " booked
the seat");
            availableSeats--;
        } else {
            System.out.println("Seat not
available for " + user);
        }
    }
}

public class BookingDemo {
    public static void main(String[] args) {
        TicketBooking booking = new
TicketBooking();

        Thread t1 = new Thread(() ->
booking.bookSeat("Amit"));
        Thread t2 = new Thread(() ->
booking.bookSeat("Rahul"));

        // TODO: Start both threads
    }
}
```

Step-by-Step Logic

Step 1: Recognize availableSeats as shared mutable state.

Step 2: The check (availableSeats > 0) and decrement must behave as one atomic critical section.

Step 3: Mark bookSeat() synchronized so only one thread can execute it on the same TicketBooking object at a time.

Step 4: Create two threads that call the same booking object.

Step 5: Start both threads and join them so main waits for completion.

Complete Java Solution

```
class TicketBooking {
    private int availableSeats = 1;

    public synchronized void bookSeat(String user) {
        if (availableSeats > 0) {
            System.out.println(user + " booked the seat");
            availableSeats--;
        } else {
            System.out.println("Seat not available for " +
user);
        }
    }
}

public class BookingDemo {
    public static void main(String[] args) throws
InterruptedException {
        TicketBooking booking = new TicketBooking();

        Thread t1 = new Thread(() ->
booking.bookSeat("Amit"));
        Thread t2 = new Thread(() ->
booking.bookSeat("Rahul"));

        t1.start();
        t2.start();

        t1.join();
        t2.join();
    }
}
```

Expected Output

Possible output:
Amit booked the seat
Seat not available for Rahul

The two names may appear in the opposite order depending on thread scheduling.

Important Interview Points

• Synchronization protects the check-and-update operation, not merely

		the variable itself. • Thread execution order is nondeterministic; correctness should not depend on which thread runs first.	

11	**Question 11: String Compression**
	Scenario
	Compress repeated consecutive characters. For example, aaabbccccdaa should become a3b2c4d1a2.
	Topics Covered
	• Strings
	• StringBuilder
	• Loops
	• Character Processing
	• String Algorithms
	Starter Code
	`public class StringCompression {`
	`  public static void main(String[] args) {`
	`  String input = "aaabbccccdaa";`
	`  StringBuilder result = new StringBuilder();`
	`  int count = 1;`
	`  for (int i = 0; i < input.length(); i++) {`
	`  // TODO: Compare current character with next character`
	`  // TODO: Increase count if same`
	`  // TODO: Append character and count when sequence changes`
	`  }`
	`  System.out.println(result);`
	`  }`
	`}`
		Step-by-Step Logic
		Step 1: Return immediately for an empty string because there is no first character to count.
		Step 2: Initialize count to 1 for the first character.
		Step 3: Start scanning from index 1 and compare the current character with the previous character.
		Step 4: If both characters are equal, increment count.
		Step 5: When they differ, append the previous character and count, then reset count to 1.
		Step 6: After the loop, append the final character group because it is not followed by another character to trigger the else block.
		Complete Java Solution
		`public class StringCompression {`
		`  public static String compress(String input) {`
		`  if (input == null || input.isEmpty()) {`
		`  return input;`
		`  }`
		`  StringBuilder result = new StringBuilder();`
		`  int count = 1;`
		`  for (int i = 1; i < input.length(); i++) {`
		`  if (input.charAt(i) == input.charAt(i - 1)) {`
		`  count++;`
		`  } else {`
		`  result.append(input.charAt(i - 1)).append(count);`
		`  count = 1;`
		`  }`
		`  }`
		`  result.append(input.charAt(input.length() - 1)).append(count);`
		`  return result.toString();`
		`  }`
		`  public static void main(String[] args) {`
		`  String input = "aaabbccccdaa";`
		`  System.out.println(compress(input));`
		`  }`
		`}`
		Expected Output
		a3b2c4d1a2
		Important Interview Points
		• StringBuilder is used because repeated String concatenation creates many temporary objects.
		• The algorithm is O(n) time and O(n) space for the output.	

12	**Question 12: Balanced Brackets**
	Scenario
	A compiler needs to validate whether brackets in an expression are properly balanced. For example, (a+b) * {c-[d/e]} is valid, while (a+b] is invalid.
	Topics Covered
	• Stack
	• Deque
	• Strings
	• Conditional Statements
	• Algorithms
	Starter Code
	`import java.util.*;`
	`public class BalancedBrackets {`
	`  public static void main(String[] args) {`
	`  String expression = "(a+b) * {c-[d/e]}";`
	`  Deque<Character> stack = new ArrayDeque<>();`
		Step-by-Step Logic
		Step 1: Use ArrayDeque<Character> as a stack. It is preferred over the legacy Stack class.
		Step 2: Push every opening bracket onto the stack.
		Step 3: When a closing bracket appears, first ensure the stack is not empty.
		Step 4: Pop the last opening bracket and verify that it matches the current closing bracket.
		Step 5: After scanning the full expression, the stack must be empty; otherwise some opening brackets were never closed.
		Complete Java Solution
		`import java.util.*;`
		`public class BalancedBrackets {`
		`  static boolean isBalanced(String expression) {`
		`  Deque<Character> stack = new ArrayDeque<>();`
		`  for (char ch : expression.toCharArray()) {`
		`  if (ch == '(' || ch == '{' || ch == '[') {`
		`  stack.push(ch);`
		`  } else if (ch == ')' || ch == '}' || ch == `	

```
        for (char ch : expression.toCharArray())
{
            // TODO: Push opening brackets
            // TODO: When a closing bracket
appears,
            // compare it with stack top
        }

        // TODO: Determine whether expression is
valid
    }
}
```

```
']') {
                if (stack.isEmpty()) {
                    return false;
                }

                char open = stack.pop();
                if (!matches(open, ch)) {
                    return false;
                }
            }
        }
        return stack.isEmpty();
    }

    static boolean matches(char open, char close) {
        return (open == '(' && close == ')') ||
               (open == '{' && close == '}') ||
               (open == '[' && close == ']');
    }

    public static void main(String[] args) {
        System.out.println(isBalanced("(a+b) * {c-
[d/e]}"));
        System.out.println(isBalanced("(a+b]"));
    }
}
```

Expected Output

true
false

Important Interview Points

• A stack is appropriate because the most recently opened bracket must be closed first (LIFO).

• Time complexity is O(n), and worst-case auxiliary space is O(n).

13

Question 13: Group Employees by Department

Scenario

An organization has employees from departments such as IT, HR, Finance, and Sales. Group employees according to department using Java Streams.

Topics Covered

• Streams
• Collectors
• groupingBy()
• Maps
• Lists
• Object Processing

Starter Code

```
import java.util.*;
import java.util.stream.*;

class Employee {
    String name;
    String department;

    Employee(String name, String department) {
        this.name = name;
        this.department = department;
    }
}

public class DepartmentGrouping {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("Amit", "IT"),
            new Employee("Rahul", "IT"),
            new Employee("Priya", "HR"),
            new Employee("Neha", "Finance")
        );

        // TODO: Use Collectors.groupingBy()
        Map<String, List<Employee>> result;

        // TODO: Display grouped employees
    }
}
```

Step-by-Step Logic

Step 1: Store Employee objects in a List.

Step 2: Create a stream from the list.

Step 3: Use Collectors.groupingBy(e -> e.department).

Step 4: The collector creates one map entry per distinct department and places matching Employee objects into that department's list.

Step 5: Traverse the resulting map to display department names and employee names.

Complete Java Solution

```
import java.util.*;
import java.util.stream.*;

class Employee {
    String name;
    String department;

    Employee(String name, String department) {
        this.name = name;
        this.department = department;
    }
}

public class DepartmentGrouping {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
                new Employee("Amit", "IT"),
                new Employee("Rahul", "IT"),
                new Employee("Priya", "HR"),
                new Employee("Neha", "Finance")
        );

        Map<String, List<Employee>> result =
employees.stream()
                .collect(Collectors.groupingBy(e ->
e.department));

        result.forEach((department, list) -> {
            System.out.print(department + " -> ");
            System.out.println(list.stream()
                    .map(e -> e.name)
                    .collect(Collectors.joining(", ")));
        });
    }
}
```

Expected Output

Example output (map order may vary):
Finance -> Neha
HR -> Priya
IT -> Amit, Rahul

Important Interview Points

• groupingBy() is one of the most common interview collectors.
• Default groupingBy uses a HashMap, so key display order is not guaranteed.

14

Question 14: Student File Processing

Scenario

A text file contains thousands of student names, one name per line. Count the number of students whose names begin with the letter A and ensure the file resource is properly closed.

Topics Covered

• File Handling
• BufferedReader
• Try-with-resources
• Exceptions
• Strings
• I/O

Starter Code

```
import java.io.*;

public class StudentFile {
    public static void main(String[] args) {
        int count = 0;

        try (BufferedReader reader =
                 new BufferedReader(new
FileReader("students.txt"))) {

            String line;
            while ((line = reader.readLine()) !=
null) {
                // TODO: Check whether name
starts with "A"
            }

        } catch (IOException e) {
            // TODO: Handle exception
        }

        // TODO: Display count
    }
}
```

Step-by-Step Logic

Step 1: Open the file through FileReader and wrap it in BufferedReader for efficient line-based reading.
Step 2: Use try-with-resources so the reader is closed automatically even if an exception occurs.
Step 3: Read lines until readLine() returns null.
Step 4: Trim each line to remove accidental leading/trailing spaces.
Step 5: Check startsWith("A") and increment the counter.
Step 6: Catch IOException and display or log the error.

Complete Java Solution

```
import java.io.*;

public class StudentFile {
    public static void main(String[] args) {
        int count = 0;

        try (BufferedReader reader =
                    new BufferedReader(new
FileReader("students.txt"))) {

            String line;
            while ((line = reader.readLine()) != null) {
                String name = line.trim();
                if (name.startsWith("A")) {
                    count++;
                }
            }

            System.out.println("Students whose names start
with A = " + count);

        } catch (IOException e) {
            System.out.println("Unable to read file: " +
e.getMessage());
        }
    }
}
```

Expected Output

For a file containing Amit, Rahul, Anjali, Priya, Akash:
Students whose names start with A = 3

Important Interview Points

• BufferedReader is memory-efficient because it processes one line at a time instead of loading the whole file.
• Try-with-resources is the preferred modern pattern for closeable resources.

15

Question 15: Payment Processing Using Method Overriding

Scenario

An application supports CreditCardPayment, UPIPayment, and NetBankingPayment. Each payment type follows a different process, but the application should call a common pay() method.

Topics Covered

• Inheritance
• Polymorphism
• Method Overriding
• Runtime Polymorphism
• OOP

Step-by-Step Logic

Step 1: Define pay(double amount) in a common parent class.
Step 2: Override pay() in each child class with payment-specific behavior.
Step 3: Declare a variable of the parent type Payment.
Step 4: Assign different child objects to that parent reference.
Step 5: When pay() is called, Java selects the overridden method based on the actual object at runtime.

Complete Java Solution

```
class Payment {
    public void pay(double amount) {
        System.out.println("Processing generic payment of
" + amount);
    }
}
```

Starter Code

```
class Payment {
    public void pay(double amount) {
        // Generic implementation
    }
}

class CreditCardPayment extends Payment {
    @Override
    public void pay(double amount) {
        // TODO: Implement credit card payment
    }
}

class UPIPayment extends Payment {
    @Override
    public void pay(double amount) {
        // TODO: Implement UPI payment
    }
}

class NetBankingPayment extends Payment {
    @Override
    public void pay(double amount) {
        // TODO: Implement net banking
    }
}

public class PaymentDemo {
    public static void main(String[] args) {
        Payment payment;

        // TODO: Assign different child objects
        // and demonstrate runtime polymorphism
    }
}
```

```
class CreditCardPayment extends Payment {
    @Override
    public void pay(double amount) {
        System.out.println("Paid " + amount + " using
Credit Card");
    }
}

class UPIPayment extends Payment {
    @Override
    public void pay(double amount) {
        System.out.println("Paid " + amount + " using
UPI");
    }
}

class NetBankingPayment extends Payment {
    @Override
    public void pay(double amount) {
        System.out.println("Paid " + amount + " using Net
Banking");
    }
}

public class PaymentDemo {
    public static void main(String[] args) {
        Payment payment;

        payment = new CreditCardPayment();
        payment.pay(1500);

        payment = new UPIPayment();
        payment.pay(800);

        payment = new NetBankingPayment();
        payment.pay(2500);
    }
}
```

Expected Output

Paid 1500.0 using Credit Card
Paid 800.0 using UPI
Paid 2500.0 using Net Banking

Important Interview Points

• This is runtime polymorphism/dynamic method dispatch.
• In a larger design, Payment could be abstract or an interface so a meaningless generic implementation is impossible.

16

Question 16: Duplicate Transaction Detection

Scenario

A payment system receives transaction IDs 101, 205, 101, 302, 205, 410, 502, 302. Find all IDs that appear more than once, using an approach suitable for very large input.

Topics Covered

• HashSet
• Collections
• Duplicate Detection
• Algorithm Complexity
• Large Datasets

Starter Code

```
import java.util.*;

public class DuplicateTransactions {
    public static void main(String[] args) {
        int[] transactions = {
            101, 205, 101, 302, 205, 410, 502,
302
        };

        Set<Integer> unique = new HashSet<>();
        Set<Integer> duplicates = new
HashSet<>();

        for (int id : transactions) {
            // TODO: Try inserting into unique
            // TODO: If already present, add it
to duplicates
        }
```

Step-by-Step Logic

Step 1: Create a HashSet named unique to record IDs seen for the first time.

Step 2: Create another Set named duplicates to store IDs that appear again.

Step 3: For each transaction ID, call unique.add(id).

Step 4: Set.add() returns false if the value is already present. In that case, add the ID to duplicates.

Step 5: Display the duplicates set. Using a Set prevents the same duplicate ID from being printed repeatedly.

Complete Java Solution

```
import java.util.*;

public class DuplicateTransactions {
    public static void main(String[] args) {
        int[] transactions = {101, 205, 101, 302, 205,
410, 502, 302};

        Set<Integer> unique = new HashSet<>();
        Set<Integer> duplicates = new LinkedHashSet<>();

        for (int id : transactions) {
            if (!unique.add(id)) {
                duplicates.add(id);
            }
        }

        System.out.println("Duplicate transaction IDs: " +
duplicates);
    }
}
```

```
        // TODO: Display duplicate IDs
    }
}
```

Expected Output

Duplicate transaction IDs: [101, 205, 302]

Important Interview Points

• Average time complexity is O(n) because HashSet insertion/lookup is O(1) on average.

• This uses O(n) extra memory; for data too large for memory, an external database/sort/streaming strategy may be needed.

17

Question 17: Order Search Using Optional

Scenario

An e-commerce application searches for an order by orderId. The order may or may not exist. Use Optional instead of returning null, and display "Order Not Found" when necessary.

Topics Covered

• Optional
• Java 8
• Null Handling
• Methods
• Object-Oriented Programming

Starter Code

```
import java.util.Optional;

class Order {
    int orderId;
    String product;

    Order(int orderId, String product) {
        this.orderId = orderId;
        this.product = product;
    }
}

public class OrderDemo {
    static Optional<Order> findOrder(int id) {
        // Assume database search
        if (id == 101) {
            return Optional.of(new Order(101,
"Laptop"));
        }

        // TODO: Return appropriate Optional
        return null;
    }

    public static void main(String[] args) {
        Optional<Order> order = findOrder(105);

        // TODO: Display order if present
        // TODO: Otherwise display "Order Not
Found"
    }
}
```

Step-by-Step Logic

Step 1: Change the method return type from Order to Optional<Order>.

Step 2: When an order is found, return Optional.of(order).

Step 3: When no order is found, return Optional.empty().

Step 4: The caller checks isPresent() (Java 8 compatible) before calling get().

Step 5: Provide an explicit else branch for the not-found case.

Complete Java Solution

```
import java.util.Optional;

class Order {
    int orderId;
    String product;

    Order(int orderId, String product) {
        this.orderId = orderId;
        this.product = product;
    }
}

public class OrderDemo {
    static Optional<Order> findOrder(int id) {
        if (id == 101) {
            return Optional.of(new Order(101, "Laptop"));
        }
        return Optional.empty();
    }

    public static void main(String[] args) {
        Optional<Order> order = findOrder(105);

        if (order.isPresent()) {
            Order value = order.get();
            System.out.println(value.orderId + " - " +
value.product);
        } else {
            System.out.println("Order Not Found");
        }
    }
}
```

Expected Output

Order Not Found

Important Interview Points

• Optional makes absence explicit in the method signature.

• Do not return null from a method whose declared return type is Optional.

18

Question 18: Producer-Consumer Log System

Scenario

One thread produces application log messages while another thread stores or processes those messages. Implement the producer-consumer mechanism using BlockingQueue.

Topics Covered

• Multithreading
• Producer-Consumer
• BlockingQueue
• Thread Communication
• Concurrent Collections

Starter Code

```
import java.util.concurrent.*;
```

Step-by-Step Logic

Step 1: Create an ArrayBlockingQueue with a fixed capacity.

Step 2: The producer uses put(). If the queue is full, put() automatically blocks until space becomes available.

Step 3: The consumer uses take(). If the queue is empty, take() automatically blocks until an item is available.

Step 4: Start both threads. No explicit synchronized, wait(), or notify() is required for queue coordination.

Step 5: Join both threads before main exits.

Complete Java Solution

```
import java.util.concurrent.*;

public class LogSystem {
    static BlockingQueue<String> queue = new
ArrayBlockingQueue<>(5);

    public static void main(String[] args) throws
```

```
public class LogSystem {
    static BlockingQueue<String> queue =
        new ArrayBlockingQueue<>(5);

    public static void main(String[] args) {
        Thread producer = new Thread(() -> {
            for (int i = 1; i <= 10; i++) {
                try {
                    // TODO: Add log message into
queue
                } catch (InterruptedException e)
{

Thread.currentThread().interrupt();
                }
            }
        });

        Thread consumer = new Thread(() -> {
            for (int i = 1; i <= 10; i++) {
                try {
                    // TODO: Retrieve message
                    // TODO: Print/store message
                } catch (InterruptedException e)
{

Thread.currentThread().interrupt();
                }
            }
        });

        // TODO: Start producer
        // TODO: Start consumer
    }
}
```

Student Task

• Explain why BlockingQueue is preferable to manually coordinating this scenario with wait() and notify().

```
InterruptedException {
        Thread producer = new Thread(() -> {
            for (int i = 1; i <= 10; i++) {
                try {
                    String message = "Log-" + i;
                    queue.put(message);
                    System.out.println("Produced: " +
message);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    return;
                }
            }
        });

        Thread consumer = new Thread(() -> {
            for (int i = 1; i <= 10; i++) {
                try {
                    String message = queue.take();
                    System.out.println("Stored:   " +
message);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    return;
                }
            }
        });

        producer.start();
        consumer.start();
        producer.join();
        consumer.join();
    }
}
```

Expected Output

The exact interleaving varies. Every Log-1 through Log-10 is produced once and consumed once.

Important Interview Points

• BlockingQueue already provides thread-safe queue operations and blocking semantics.

• Restoring the interrupt flag with Thread.currentThread().interrupt() is a good practice when catching InterruptedException.

19

Question 19: Highest-Paid Employee per Department

Scenario

Given employees with name, department, and salary, use Java Streams to find the highest-paid employee from each department.

Topics Covered

• Streams
• Collectors.groupingBy()
• maxBy()
• Comparator
• Maps
• Optional

Starter Code

```
import java.util.*;
import java.util.stream.*;

class Employee {
    String name;
    String department;
    double salary;

    Employee(String name, String department,
double salary) {
        this.name = name;
        this.department = department;
        this.salary = salary;
    }
}

public class DepartmentSalary {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
```

Step-by-Step Logic

Step 1: Group employees by department with groupingBy().

Step 2: As the downstream collector, use maxBy() with a salary comparator.

Step 3: Because a maximum may conceptually be absent, maxBy() produces Optional<Employee> for each group.

Step 4: Traverse the resulting map and unwrap each Optional when printing.

Step 5: No manual nested loops are required.

Complete Java Solution

```
import java.util.*;
import java.util.stream.*;

class Employee {
    String name;
    String department;
    double salary;

    Employee(String name, String department, double
salary) {
        this.name = name;
        this.department = department;
        this.salary = salary;
    }
}

public class DepartmentSalary {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
                new Employee("Amit", "IT", 70000),
                new Employee("Rahul", "IT", 90000),
                new Employee("Priya", "HR", 75000),
                new Employee("Neha", "HR", 68000),
                new Employee("Raj", "Sales", 85000)
        );
```

```
            new Employee("Amit", "IT", 70000),
            new Employee("Rahul", "IT", 90000),
            new Employee("Priya", "HR", 75000),
            new Employee("Neha", "HR", 68000),
            new Employee("Raj", "Sales", 85000)
        );

        // TODO: Group employees by department
        // TODO: Find maximum salary in every
department
        // TODO: Display result
    }
}
```

```
        Map<String, Optional<Employee>> highestPaid =
employees.stream()
                .collect(Collectors.groupingBy(
                        e -> e.department,

Collectors.maxBy(Comparator.comparingDouble(e ->
e.salary))));

        highestPaid.forEach((department, employeeOpt) -> {
            Employee e = employeeOpt.get();
            System.out.println(department + " -> " +
e.name + " - " + e.salary);
        });
    }
}
```

Expected Output

Example output (department order may vary):
Sales -> Raj - 85000.0
HR -> Priya - 75000.0
IT -> Rahul - 90000.0

Important Interview Points

• groupingBy + maxBy is a standard stream pattern for 'best item per category'.

• In production code, avoid Optional.get() unless presence is guaranteed by the grouping logic or checked explicitly.

20

Question 20: Library Management System

Scenario

A library contains books that can be added, searched, issued, and returned. Multiple users may search for books, but only one user should be able to issue a particular available copy at a time.

Topics Covered

• OOP
• Classes and Objects
• Encapsulation
• Collections
• Searching
• Methods
• Synchronization
• Real-World Application Design

Starter Code

```
import java.util.*;

class Book {
    private int bookId;
    private String title;
    private String author;
    private boolean available = true;

    Book(int bookId, String title, String author)
{
        this.bookId = bookId;
        this.title = title;
        this.author = author;
    }

    public String getTitle() {
        return title;
    }

    public boolean isAvailable() {
        return available;
    }

    // TODO: Create method to issue book
    // TODO: Create method to return book
}

class Library {
    List<Book> books = new ArrayList<>();

    public void addBook(Book book) {
        // TODO
```

Step-by-Step Logic

Step 1: Represent each physical book copy as a Book object with an available flag.

Step 2: Keep the books in a List inside Library.

Step 3: Provide searchBook() to find matching titles.

Step 4: Place the critical availability check and state change inside synchronized Book.issue(). This locks only the selected book rather than the whole library.

Step 5: Book.returnBook() changes the same shared state in a synchronized method.

Step 6: Library.issueBook() searches for the book and delegates the atomic issue operation to Book.

Complete Java Solution

```
import java.util.*;

class Book {
    private final int bookId;
    private final String title;
    private final String author;
    private boolean available = true;

    Book(int bookId, String title, String author) {
        this.bookId = bookId;
        this.title = title;
        this.author = author;
    }

    public String getTitle() {
        return title;
    }

    public synchronized boolean issue() {
        if (!available) {
            return false;
        }
        available = false;
        return true;
    }

    public synchronized void returnBook() {
        available = true;
    }

    public synchronized boolean isAvailable() {
        return available;
    }

    @Override
    public String toString() {
        return bookId + " - " + title + " by " + author +
```

```
    }

    public void searchBook(String title) {
        // TODO
    }

    public void issueBook(String title) {
        // TODO: Make the critical operation
thread-safe
    }

    public void returnBook(String title) {
        // TODO
    }
}

public class LibraryDemo {
    public static void main(String[] args) {
        Library library = new Library();

        // TODO: Add sample books
        // TODO: Search book
        // TODO: Issue book
        // TODO: Return book
    }
}
```

```
                " [" + (available ? "Available" :
"Issued") + "]";
    }
}

class Library {
    private final List<Book> books = new ArrayList<>();

    public void addBook(Book book) {
        books.add(book);
    }

    public Book searchBook(String title) {
        for (Book book : books) {
            if (book.getTitle().equalsIgnoreCase(title)) {
                return book;
            }
        }
        return null;
    }

    public void issueBook(String title, String user) {
        Book book = searchBook(title);
        if (book == null) {
            System.out.println("Book not found");
        } else if (book.issue()) {
            System.out.println(user + " issued " + title);
        } else {
            System.out.println(title + " is already
issued");
        }
    }

    public void returnBook(String title) {
        Book book = searchBook(title);
        if (book != null) {
            book.returnBook();
            System.out.println(title + " returned");
        }
    }
}

public class LibraryDemo {
    public static void main(String[] args) throws
InterruptedException {
        Library library = new Library();
        library.addBook(new Book(1, "Java Basics", "A.
Author"));
        library.addBook(new Book(2, "Data Structures", "B.
Author"));

        Thread t1 = new Thread(() ->
library.issueBook("Java Basics", "Amit"));
        Thread t2 = new Thread(() ->
library.issueBook("Java Basics", "Priya"));

        t1.start();
        t2.start();
        t1.join();
        t2.join();

        library.returnBook("Java Basics");
    }
}
```

Expected Output

Possible output:
Amit issued Java Basics
Java Basics is already issued
Java Basics returned

Amit/Priya may exchange positions depending on thread scheduling.

Important Interview Points

• Synchronizing the Book itself gives finer-grained locking than synchronizing every Library operation.

• A production system would identify book copies by unique ID and would typically persist issue records in a database transaction.

21	**Question 21: Notification System Using Interface** **Scenario** An application sends notifications through Email, SMS,	**Step-by-Step Logic** Step 1: Declare the behavior sendNotification() in an interface. Step 2: Create one implementation class for each notification channel.	

and Push Notification. Every notification mechanism should implement a common sendNotification() operation.

Topics Covered

- Interfaces
- Interface Implementation
- Abstraction
- Polymorphism
- Method Overriding
- Loose Coupling

Starter Code

```
interface Notification {
    void sendNotification(String message);
}

class EmailNotification implements Notification {
    @Override
    public void sendNotification(String message)
{
        // TODO: Display email notification
    }
}

class SMSNotification implements Notification {
    @Override
    public void sendNotification(String message)
{
        // TODO: Display SMS notification
    }
}

class PushNotification implements Notification {
    @Override
    public void sendNotification(String message)
{
        // TODO: Display push notification
    }
}

public class NotificationDemo {
    public static void main(String[] args) {
        Notification notification;

        // TODO: Create EmailNotification object
and send notification
        // TODO: Repeat for SMS and Push
Notification
    }
}
```

Student Task

- Modify the program so that the notification type can be selected at runtime.

Step 3: Use a Notification reference instead of a concrete type in the main workflow.

Step 4: Select the required implementation at runtime using input or configuration.

Step 5: Invoke sendNotification() through the interface reference.

Complete Java Solution

```
import java.util.Scanner;

interface Notification {
    void sendNotification(String message);
}

class EmailNotification implements Notification {
    public void sendNotification(String message) {
        System.out.println("EMAIL: " + message);
    }
}

class SMSNotification implements Notification {
    public void sendNotification(String message) {
        System.out.println("SMS: " + message);
    }
}

class PushNotification implements Notification {
    public void sendNotification(String message) {
        System.out.println("PUSH: " + message);
    }
}

public class NotificationDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Choose EMAIL, SMS, or PUSH: ");
        String choice =
scanner.nextLine().trim().toUpperCase();

        Notification notification;
        switch (choice) {
            case "EMAIL":
                notification = new EmailNotification();
                break;
            case "SMS":
                notification = new SMSNotification();
                break;
            case "PUSH":
                notification = new PushNotification();
                break;
            default:
                System.out.println("Invalid notification
type");
                scanner.close();
                return;
        }

        notification.sendNotification("Your order has been
shipped.");
        scanner.close();
    }
}
```

Expected Output

If the user enters SMS:
SMS: Your order has been shipped.

Important Interview Points

- Interfaces separate what an object can do from how it does it.
- The caller depends on Notification, so new channels can be added with less change to existing code.

22	**Question 22: Employee Salary Using Abstract Class** **Scenario** A company has Permanent Employees and Contract Employees. Both have a name and ID, but their salary calculations are different. **Topics Covered** • Abstract Classes	**Step-by-Step Logic** Step 1: Put shared fields id and name in an abstract Employee class. Step 2: Provide a normal displayEmployee() method because its behavior is common. Step 3: Declare calculateSalary() abstract because each employee type calculates salary differently. Step 4: Override calculateSalary() in PermanentEmployee and ContractEmployee. Step 5: Store child objects in Employee references and call	

• Abstract Methods
• Inheritance
• Method Overriding
• Polymorphism

Starter Code

```
abstract class Employee {
    int id;
    String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    abstract double calculateSalary();

    void displayEmployee() {
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
    }
}

class PermanentEmployee extends Employee {
    double basicSalary;

    PermanentEmployee(int id, String name, double
basicSalary) {
        super(id, name);
        this.basicSalary = basicSalary;
    }

    @Override
    double calculateSalary() {
        // TODO: Salary = basic salary +
allowances
        return 0;
    }
}

class ContractEmployee extends Employee {
    int hoursWorked;
    double ratePerHour;

    ContractEmployee(int id, String name,
                     int hoursWorked, double
ratePerHour) {
        super(id, name);
        this.hoursWorked = hoursWorked;
        this.ratePerHour = ratePerHour;
    }

    @Override
    double calculateSalary() {
        // TODO: Calculate salary
        return 0;
    }
}

public class EmployeeDemo {
    public static void main(String[] args) {
        // TODO: Create both employee types
        // TODO: Display salaries using Employee
reference
    }
}
```

calculateSalary() polymorphically.

Complete Java Solution

```
abstract class Employee {
    int id;
    String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    abstract double calculateSalary();

    void displayEmployee() {
        System.out.println(id + " - " + name);
    }
}

class PermanentEmployee extends Employee {
    double basicSalary;

    PermanentEmployee(int id, String name, double
basicSalary) {
        super(id, name);
        this.basicSalary = basicSalary;
    }

    @Override
    double calculateSalary() {
        double allowance = basicSalary * 0.20; //
Assumption: 20% allowance
        return basicSalary + allowance;
    }
}

class ContractEmployee extends Employee {
    int hoursWorked;
    double ratePerHour;

    ContractEmployee(int id, String name, int hoursWorked,
double ratePerHour) {
        super(id, name);
        this.hoursWorked = hoursWorked;
        this.ratePerHour = ratePerHour;
    }

    @Override
    double calculateSalary() {
        return hoursWorked * ratePerHour;
    }
}

public class EmployeeDemo {
    public static void main(String[] args) {
        Employee e1 = new PermanentEmployee(101, "Amit",
50000);
        Employee e2 = new ContractEmployee(102, "Priya",
160, 400);

        e1.displayEmployee();
        System.out.println("Salary = " +
e1.calculateSalary());

        e2.displayEmployee();
        System.out.println("Salary = " +
e2.calculateSalary());
    }
}
```

Expected Output

101 - Amit
Salary = 60000.0
102 - Priya
Salary = 64000.0

Important Interview Points

• Use an abstract class when subclasses share both state/implementation and a required abstract behavior.
• The 20% allowance is an explicit example assumption; real business rules should come from requirements.

23	**Question 23: Generic Repository**	**Step-by-Step Logic**	

Scenario

A software application stores different object types such as Students, Employees, and Products. Instead of writing separate storage classes, design one generic repository.

Topics Covered

- Generics
- Generic Classes
- Type Parameters
- Collections
- Reusable Code

Starter Code

```
import java.util.*;

class Repository<T> {
    private List<T> items = new ArrayList<>();

    public void add(T item) {
        // TODO: Add item
    }

    public void remove(T item) {
        // TODO: Remove item
    }

    public List<T> getAll() {
        // TODO: Return stored items
        return null;
    }
}

class Student {
    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
}

public class GenericDemo {
    public static void main(String[] args) {
        Repository<Student> students = new
Repository<>();

        // TODO: Add students
        // TODO: Retrieve and display them
    }
}
```

Step-by-Step Logic

Step 1: Declare the class as Repository<T>. T is a placeholder for the element type.

Step 2: Use List<T> internally, so the repository only accepts objects of the selected type.

Step 3: Implement add(T), remove(T), and getAll().

Step 4: Instantiate Repository<Student> when storing students.

Step 5: The compiler now rejects an attempt to add an unrelated type into the student repository.

Complete Java Solution

```
import java.util.*;

class Repository<T> {
    private final List<T> items = new ArrayList<>();

    public void add(T item) {
        items.add(item);
    }

    public void remove(T item) {
        items.remove(item);
    }

    public List<T> getAll() {
        return new ArrayList<>(items);
    }
}

class Student {
    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    @Override
    public String toString() {
        return id + " - " + name;
    }
}

public class GenericDemo {
    public static void main(String[] args) {
        Repository<Student> students = new Repository<>();
        students.add(new Student(1, "Amit"));
        students.add(new Student(2, "Priya"));

        for (Student student : students.getAll()) {
            System.out.println(student);
        }
    }
}
```

Expected Output

1 - Amit
2 - Priya

Important Interview Points

- Generics provide compile-time type safety and reduce casts.
- Returning a new ArrayList protects the repository's internal List from direct external modification.

24

Question 24: Generic Calculator with Bounded Types

Scenario

A calculator should accept numerical values such as Integer, Double, and Float but reject values such as String.

Topics Covered

- Generic Methods
- Bounded Generics
- Number
- Type Safety

Starter Code

```
class Calculator {
    public static <T extends Number>
```

Step-by-Step Logic

Step 1: Declare the type parameter as <T extends Number>.

Step 2: The bound tells the compiler that T must be Number or one of its subclasses.

Step 3: Number provides conversion methods such as doubleValue().

Step 4: Convert both operands to double and add them.

Step 5: Trying Calculator.add("10", "20") fails at compile time because String does not extend Number.

Complete Java Solution

```
class Calculator {
    public static <T extends Number> double add(T number1,
T number2) {
        return number1.doubleValue() +
number2.doubleValue();
```

```
    double add(T number1, T number2) {
        // TODO: Convert values to double
        // TODO: Return sum
        return 0;
    }
}

public class CalculatorDemo {
    public static void main(String[] args) {
        // TODO: Add two integers
        // TODO: Add two doubles

        // Experiment:
        // Try passing Strings and observe the
compiler response.
    }
}
```

```
    }
}

public class CalculatorDemo {
    public static void main(String[] args) {
        System.out.println(Calculator.add(10, 20));
        System.out.println(Calculator.add(5.5, 2.25));
        System.out.println(Calculator.add(3.5f, 1.5f));

        // Calculator.add("10", "20"); // Compile-time
error
    }
}
```

Expected Output

30.0
7.75
5.0

Important Interview Points

• A bounded type parameter lets generic code safely call methods defined by the bound.

• For precise financial arithmetic, BigDecimal is usually preferable to floating-point arithmetic.

25

Question 25: Order Status Management Using Enum

Scenario

An online order can have the states PLACED, CONFIRMED, SHIPPED, DELIVERED, or CANCELLED. Create an order-processing system using an enum.

Topics Covered

• Enum
• switch
• Classes
• State Representation
• Constants

Starter Code

```
enum OrderStatus {
    PLACED,
    CONFIRMED,
    SHIPPED,
    DELIVERED,
    CANCELLED
}

class Order {
    int orderId;
    OrderStatus status;

    Order(int orderId) {
        this.orderId = orderId;
        this.status = OrderStatus.PLACED;
    }

    void updateStatus(OrderStatus newStatus) {
        // TODO: Update status
    }

    void displayStatus() {
        // TODO: Use switch to display suitable
message
    }
}

public class OrderStatusDemo {
    public static void main(String[] args) {
        Order order = new Order(101);

        // TODO: Change order status
        // TODO: Display current status
    }
}
```

Step-by-Step Logic

Step 1: Declare every legal status in the OrderStatus enum.

Step 2: Store OrderStatus as the status field inside Order.

Step 3: Initialize a new order to PLACED.

Step 4: Update the state through updateStatus().

Step 5: Use switch(status) to select a clear message for the current state.

Complete Java Solution

```
enum OrderStatus {
    PLACED,
    CONFIRMED,
    SHIPPED,
    DELIVERED,
    CANCELLED
}

class Order {
    int orderId;
    OrderStatus status;

    Order(int orderId) {
        this.orderId = orderId;
        this.status = OrderStatus.PLACED;
    }

    void updateStatus(OrderStatus newStatus) {
        this.status = newStatus;
    }

    void displayStatus() {
        switch (status) {
            case PLACED:
                System.out.println("Order has been
placed.");
                break;
            case CONFIRMED:
                System.out.println("Order has been
confirmed.");
                break;
            case SHIPPED:
                System.out.println("Order is on the
way.");
                break;
            case DELIVERED:
                System.out.println("Order delivered
successfully.");
                break;
            case CANCELLED:
                System.out.println("Order has been
cancelled.");
                break;
        }
    }
}
```

```
public class OrderStatusDemo {
    public static void main(String[] args) {
        Order order = new Order(101);
        order.displayStatus();

        order.updateStatus(OrderStatus.SHIPPED);
        order.displayStatus();
    }
}
```

Expected Output

Order has been placed.
Order is on the way.

Important Interview Points

• Enum prevents invalid values such as 'shiped' that can occur with raw strings.

• More advanced systems may validate which state transitions are legal.

26

Question 26: Student Record Using Java Record

Scenario

A college wants a lightweight immutable object to store roll number, name, department, and percentage. Use a Java record instead of a traditional class.

Topics Covered

• Java Records
• Immutable Data
• Modern Java
• Constructors
• Accessor Methods

Starter Code

```
record Student(
        int rollNumber,
        String name,
        String department,
        double percentage) {
}

public class RecordDemo {
    public static void main(String[] args) {
        Student s1 = new Student(101, "Amit",
"CSE", 82.5);
        Student s2 = new Student(102, "Priya",
"CSE", 91.3);

        // TODO: Display student information
        // TODO: Compare two records
        // TODO: Display automatically generated
toString()
    }
}
```

Student Task

• What methods does Java automatically provide for a record?

Step-by-Step Logic

Step 1: Declare record Student(...) with the required components.

Step 2: Java automatically creates private final fields and a canonical constructor.

Step 3: Access values through methods named after the components, such as s1.name() and s1.percentage().

Step 4: Use equals() to compare record values; Java automatically implements value-based equality.

Step 5: Use toString(); Java generates a readable representation automatically.

Complete Java Solution

```
record Student(int rollNumber, String name, String
department, double percentage) {
}

public class RecordDemo {
    public static void main(String[] args) {
        Student s1 = new Student(101, "Amit", "CSE",
82.5);
        Student s2 = new Student(102, "Priya", "CSE",
91.3);
        Student s3 = new Student(101, "Amit", "CSE",
82.5);

        System.out.println("Name: " + s1.name());
        System.out.println("Percentage: " +
s1.percentage());
        System.out.println("s1 equals s2? " +
s1.equals(s2));
        System.out.println("s1 equals s3? " +
s1.equals(s3));
        System.out.println(s1);
    }
}
```

Expected Output

Name: Amit
Percentage: 82.5
s1 equals s2? false
s1 equals s3? true
Student[rollNumber=101, name=Amit, department=CSE, percentage=82.5]

Important Interview Points

• Records automatically provide component accessors, equals(), hashCode(), and toString().

• Records became a permanent Java feature in Java 16; use an appropriate JDK when compiling this question.

27

Question 27: Calculate Employee Experience Using Date and Time API

Scenario

An employee joined a company on 15 June 2020. Calculate the employee's years, months, and days of service using the modern Java Date/Time API.

Step-by-Step Logic

Step 1: Represent the joining date using LocalDate.of().

Step 2: Obtain today's date using LocalDate.now().

Step 3: Use Period.between(joiningDate, currentDate) to calculate a calendar-based difference.

Step 4: Read years, months, and days through getYears(), getMonths(), and getDays().

Topics Covered
• LocalDate
• Period
• Java Date/Time API
• Modern Date Handling

Starter Code

```
import java.time.*;

public class ExperienceCalculator {
    public static void main(String[] args) {
        LocalDate joiningDate =
LocalDate.of(2020, 6, 15);
        LocalDate currentDate = LocalDate.now();

        // TODO: Calculate difference using
Period
        // TODO: Display years
        // TODO: Display months
        // TODO: Display days
    }
}
```

Step 5: Remember that the result changes as the current date changes.

Complete Java Solution

```
import java.time.*;

public class ExperienceCalculator {
    public static void main(String[] args) {
        LocalDate joiningDate = LocalDate.of(2020, 6, 15);
        LocalDate currentDate = LocalDate.now();

        Period experience = Period.between(joiningDate,
currentDate);

        System.out.println("Joining date: " +
joiningDate);
        System.out.println("Current date: " +
currentDate);
        System.out.println("Experience: " +
experience.getYears() + " years, "
                + experience.getMonths() + " months, "
                + experience.getDays() + " days");
    }
}
```

Expected Output

The output depends on the date the program runs.
For 21 September 2026, the experience is 6 years, 3 months, 6 days.

Important Interview Points

• java.time classes are immutable and thread-safe.
• Period is calendar-based; Duration is better for time-based differences measured in seconds/nanoseconds.

28

Question 28: Functional Interfaces for Employee Filtering

Scenario

HR wants to apply different employee-selection rules such as salary > 50000 or department = IT. Use a functional interface instead of writing separate filtering methods for each rule.

Topics Covered
• Functional Interfaces
• Predicate
• Lambda Expressions
• Java 8 Functional Programming
• Collections

Starter Code

```
import java.util.*;
import java.util.function.*;

class Employee {
    String name;
    String department;
    double salary;

    Employee(String name, String department,
double salary) {
        this.name = name;
        this.department = department;
        this.salary = salary;
    }
}

public class EmployeeFilter {
    static void filterEmployees(
            List<Employee> employees,
            Predicate<Employee> condition) {

        for (Employee e : employees) {
            // TODO: Apply condition
            // TODO: Display matching employee
        }
    }

    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("Amit", "IT", 70000),
            new Employee("Rahul", "HR", 45000),
```

Step-by-Step Logic

Step 1: Accept Predicate<Employee> as a parameter. Predicate represents a function that receives an Employee and returns boolean.
Step 2: Inside filterEmployees(), call condition.test(employee).
Step 3: Create one lambda for salary filtering.
Step 4: Create another lambda for department filtering.
Step 5: Pass either Predicate to the same method, avoiding duplicated loops.

Complete Java Solution

```
import java.util.*;
import java.util.function.*;

class Employee {
    String name;
    String department;
    double salary;

    Employee(String name, String department, double
salary) {
        this.name = name;
        this.department = department;
        this.salary = salary;
    }
}

public class EmployeeFilter {
    static void filterEmployees(List<Employee> employees,
Predicate<Employee> condition) {
        for (Employee e : employees) {
            if (condition.test(e)) {
                System.out.println(e.name + " - " +
e.department + " - " + e.salary);
            }
        }
    }

    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
                new Employee("Amit", "IT", 70000),
                new Employee("Rahul", "HR", 45000),
                new Employee("Priya", "IT", 80000)
        );

        Predicate<Employee> highSalary = e -> e.salary >
50000;
        Predicate<Employee> itDepartment = e ->
e.department.equals("IT");
```

```
            new Employee("Priya", "IT", 80000)
        );

        // TODO: Predicate for salary > 50000
        // TODO: Predicate for IT department
        // TODO: Call filterEmployees()
    }
}
```

```
        System.out.println("Salary > 50000:");
        filterEmployees(employees, highSalary);

        System.out.println("IT Department:");
        filterEmployees(employees, itDepartment);
    }
}
```

Expected Output

Salary > 50000:
Amit - IT - 70000.0
Priya - IT - 80000.0
IT Department:
Amit - IT - 70000.0
Priya - IT - 80000.0

Important Interview Points

• Predicate<T> has the abstract method boolean test(T value).
• Predicates can also be combined with and(), or(), and negate().

29

Question 29: Product Processing Using Function, Consumer, and Supplier

Scenario

An e-commerce application should generate a default product, calculate a discounted price, and display product information using standard Java functional interfaces.

Topics Covered

• Supplier
• Function
• Consumer
• Lambda Expressions
• Functional Programming

Starter Code

```
import java.util.function.*;

class Product {
    String name;
    double price;

    Product(String name, double price) {
        this.name = name;
        this.price = price;
    }
}

public class FunctionalInterfaceDemo {
    public static void main(String[] args) {
        Supplier<Product> supplier = () -> {
            // TODO: Return default product
            return null;
        };

        Function<Product, Double>
discountCalculator = product -> {
            // TODO: Calculate 10% discount
            return 0.0;
        };

        Consumer<Product> display = product -> {
            // TODO: Display product details
        };

        // TODO: Obtain product from supplier
        // TODO: Calculate discounted price
        // TODO: Display product
    }
}
```

Step-by-Step Logic

Step 1: Use Supplier<Product> for an operation that takes no input and supplies a Product.
Step 2: Use Function<Product,Double> because it receives a Product and returns a calculated Double.
Step 3: Use Consumer<Product> because it receives a Product and performs an action without returning a value.
Step 4: Call supplier.get(), function.apply(product), and consumer.accept(product).
Step 5: This demonstrates three common functional interface shapes.

Complete Java Solution

```
import java.util.function.*;

class Product {
    String name;
    double price;

    Product(String name, double price) {
        this.name = name;
        this.price = price;
    }
}

public class FunctionalInterfaceDemo {
    public static void main(String[] args) {
        Supplier<Product> supplier = () -> new
Product("Laptop", 60000);

        Function<Product, Double> discountCalculator =
                product -> product.price * 0.90; // 10%
discount

        Consumer<Product> display =
                product -> System.out.println(product.name
+ " original price = " + product.price);

        Product product = supplier.get();
        display.accept(product);

        double discountedPrice =
discountCalculator.apply(product);
        System.out.println("Discounted price = " +
discountedPrice);
    }
}
```

Expected Output

Laptop original price = 60000.0
Discounted price = 54000.0

Important Interview Points

• Supplier: no input, returns a value. Function: input -> output. Consumer: input -> side effect/no return.
• These interfaces are widely used by the Stream API and callback-style code.

30

Question 30: Student Sorting Using Method References

Step-by-Step Logic

Step 1: Create a stream from the names list.

Scenario

A college wants to display student names alphabetically. Use method references where appropriate instead of unnecessarily long lambda expressions.

Topics Covered

• Method References
• Streams
• Lambda Simplification
• Collections
• Sorting

Starter Code

```
import java.util.*;

public class MethodReferenceDemo {
    public static void main(String[] args) {
        List<String> students = Arrays.asList(
            "Rahul", "Amit", "Neha", "Priya",
"Akash"
        );

        // TODO: Sort student names
        // TODO: Display names using method
reference

        // Hint:
        // System.out::println
    }
}
```

Student Task

• Rewrite name -> System.out.println(name) using a method reference.

Step 2: Call sorted() to apply the natural String order.

Step 3: forEach() expects a Consumer<String>.

Step 4: System.out.println already has the required one-argument behavior, so replace name -> System.out.println(name) with System.out::println.

Step 5: The method reference is shorter but semantically equivalent to the lambda.

Complete Java Solution

```
import java.util.*;

public class MethodReferenceDemo {
    public static void main(String[] args) {
        List<String> students = Arrays.asList(
                "Rahul", "Amit", "Neha", "Priya",
"Akash");

        students.stream()
                .sorted()
                .forEach(System.out::println);
    }
}
```

Expected Output

Akash
Amit
Neha
Priya
Rahul

Important Interview Points

• name -> System.out.println(name) and System.out::println are equivalent here.

• Common method-reference forms are object::instanceMethod, Class::staticMethod, Class::instanceMethod, and Class::new.

31

Question 31: Save and Restore Student Objects

Scenario

A college application should save a Student object to a file before the program closes and restore it when the application starts again.

Topics Covered

• Serialization
• Deserialization
• Serializable
• ObjectOutputStream
• ObjectInputStream
• File Handling

Starter Code

```
import java.io.*;

class Student implements Serializable {
    private static final long serialVersionUID =
1L;

    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
}

public class SerializationDemo {
    public static void main(String[] args) {
        Student student = new Student(101,
"Amit");

        // Serialization
        try (ObjectOutputStream out =
                new ObjectOutputStream(
                    new
FileOutputStream("student.dat"))) {

            // TODO: Write student object
```

Step-by-Step Logic

Step 1: Make Student implement Serializable.

Step 2: Define serialVersionUID so class-version compatibility is explicit.

Step 3: Use ObjectOutputStream and writeObject(student) to serialize the object.

Step 4: Use try-with-resources so streams close automatically.

Step 5: Open ObjectInputStream and call readObject().

Step 6: Cast the returned Object to Student and display its values.

Complete Java Solution

```
import java.io.*;

class Student implements Serializable {
    private static final long serialVersionUID = 1L;

    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
}

public class SerializationDemo {
    public static void main(String[] args) {
        Student student = new Student(101, "Amit");

        try (ObjectOutputStream out = new
ObjectOutputStream(
                new FileOutputStream("student.dat"))) {
            out.writeObject(student);
            System.out.println("Student serialized.");
        } catch (IOException e) {
            System.out.println("Serialization error: " +
e.getMessage());
        }

        try (ObjectInputStream in = new ObjectInputStream(
                new FileInputStream("student.dat"))) {
            Student restored = (Student) in.readObject();
```

```
        } catch (IOException e) {
            // TODO: Handle exception
        }

        // Deserialization
        try (ObjectInputStream in =
                new ObjectInputStream(
                    new
FileInputStream("student.dat"))) {

            // TODO: Read student object
            // TODO: Display restored object

        } catch (IOException |
ClassNotFoundException e) {
            // TODO: Handle exception
        }
    }
}
```

```
            System.out.println("Restored student: " +
restored.id + " - " + restored.name);
        } catch (IOException | ClassNotFoundException e) {
            System.out.println("Deserialization error: " +
e.getMessage());
        }
    }
}
```

Expected Output

Student serialized.
Restored student: 101 - Amit

Important Interview Points

• Java native serialization is useful for learning, but many modern applications prefer JSON/other explicit formats for long-term data interchange.

• Fields marked transient are not serialized.

32

Question 32: Order Processing Using ExecutorService

Scenario

An e-commerce company receives many orders simultaneously. Instead of creating a new Thread manually for every order, process the orders using a fixed-size thread pool.

Topics Covered

• ExecutorService
• Thread Pools
• Concurrency
• Runnable Tasks
• Resource Management

Starter Code

```
import java.util.concurrent.*;

class OrderTask implements Runnable {
    int orderId;

    OrderTask(int orderId) {
        this.orderId = orderId;
    }

    @Override
    public void run() {
        // TODO: Display order being processed
        // TODO: Display current thread name
    }
}

public class OrderProcessor {
    public static void main(String[] args) {
        ExecutorService executor =
            Executors.newFixedThreadPool(3);

        for (int i = 1; i <= 10; i++) {
            // TODO: Submit OrderTask
        }

        // TODO: Shutdown executor
    }
}
```

Student Task

• Why can a thread pool be better than creating 1,000 separate threads?

Step-by-Step Logic

Step 1: Create a Runnable task class containing the order ID.

Step 2: Create ExecutorService with Executors.newFixedThreadPool(3).

Step 3: Submit each OrderTask to the executor.

Step 4: The executor reuses at most three worker threads to process all tasks.

Step 5: Call shutdown() after submissions so no new tasks are accepted.

Step 6: Optionally call awaitTermination() when main needs to wait for all tasks to finish.

Complete Java Solution

```
import java.util.concurrent.*;

class OrderTask implements Runnable {
    private final int orderId;

    OrderTask(int orderId) {
        this.orderId = orderId;
    }

    @Override
    public void run() {
        System.out.println("Processing order " + orderId +
                " by " +
Thread.currentThread().getName());
    }
}

public class OrderProcessor {
    public static void main(String[] args) throws
InterruptedException {
        ExecutorService executor =
Executors.newFixedThreadPool(3);

        for (int i = 1; i <= 10; i++) {
            executor.submit(new OrderTask(i));
        }

        executor.shutdown();
        if (!executor.awaitTermination(5,
TimeUnit.SECONDS)) {
            executor.shutdownNow();
        }
    }
}
```

Expected Output

Ten lines are printed. The order and worker-thread assignment may vary, for example:
Processing order 1 by pool-1-thread-1
Processing order 2 by pool-1-thread-2
Processing order 3 by pool-1-thread-3
...

Important Interview Points

• Thread pools avoid the overhead and resource risk of creating an unbounded number of threads.

• shutdown() is important; otherwise executor threads can keep the JVM alive.

33

Question 33: Database Employee Management Using JDBC

Scenario

A company stores employee records in a relational database. Create Java code to connect to the database, insert an employee, retrieve employees, and update employee salary.

Topics Covered

• JDBC
• Database Connection
• Connection
• PreparedStatement
• ResultSet
• SQL
• Exception Handling
• Try-with-resources

Starter Code

```
import java.sql.*;

public class EmployeeDatabase {
    public static void main(String[] args) {
        String url =
"jdbc:mysql://localhost:3306/company";
        String username = "root";
        String password = "password";

        String sql =
            "INSERT INTO employee (id, name,
salary) VALUES (?, ?, ?)";

        try (Connection connection =
                DriverManager.getConnection(url,
username, password);
             PreparedStatement statement =

connection.prepareStatement(sql)) {

            // TODO: Set employee ID
            // TODO: Set employee name
            // TODO: Set salary
            // TODO: Execute statement

        } catch (SQLException e) {
            // TODO: Handle database exception
        }
    }
}
```

Student Task

• Implement searchEmployee().
• Implement updateSalary().
• Implement deleteEmployee().
• Implement displayAllEmployees().

Step-by-Step Logic

Step 1: Create the employee table once in the database.

Step 2: Open a JDBC Connection with DriverManager.getConnection().

Step 3: Use PreparedStatement rather than string concatenation. It handles values safely and helps prevent SQL injection.

Step 4: For INSERT/UPDATE/DELETE, bind parameters and call executeUpdate().

Step 5: For SELECT, call executeQuery() and iterate through ResultSet.

Step 6: Use try-with-resources for Connection, PreparedStatement, and ResultSet so database resources are closed reliably.

Complete Java Solution

```
import java.sql.*;

public class EmployeeDatabase {
    private static final String URL =
"jdbc:mysql://localhost:3306/company";
    private static final String USER = "root";
    private static final String PASSWORD = "password";

    static Connection getConnection() throws SQLException
{
        return DriverManager.getConnection(URL, USER,
PASSWORD);
    }

    static void insertEmployee(int id, String name, double
salary) throws SQLException {
        String sql = "INSERT INTO employee(id, name,
salary) VALUES (?, ?, ?)";
        try (Connection con = getConnection();
             PreparedStatement ps =
con.prepareStatement(sql)) {
            ps.setInt(1, id);
            ps.setString(2, name);
            ps.setDouble(3, salary);
            System.out.println(ps.executeUpdate() + "
employee inserted.");
        }
    }

    static void searchEmployee(int id) throws SQLException
{
        String sql = "SELECT id, name, salary FROM
employee WHERE id = ?";
        try (Connection con = getConnection();
             PreparedStatement ps =
con.prepareStatement(sql)) {
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    System.out.println(rs.getInt("id") + "
- " +
                                       rs.getString("name") + " - " +
rs.getDouble("salary"));
                } else {
                    System.out.println("Employee not
found.");
                }
            }
        }
    }

    static void updateSalary(int id, double salary) throws
SQLException {
        String sql = "UPDATE employee SET salary = ? WHERE
id = ?";
        try (Connection con = getConnection();
             PreparedStatement ps =
con.prepareStatement(sql)) {
            ps.setDouble(1, salary);
            ps.setInt(2, id);
            System.out.println(ps.executeUpdate() + "
employee updated.");
        }
    }
```

```
    static void deleteEmployee(int id) throws SQLException
{
        String sql = "DELETE FROM employee WHERE id = ?";
        try (Connection con = getConnection();
             PreparedStatement ps =
con.prepareStatement(sql)) {
            ps.setInt(1, id);
            System.out.println(ps.executeUpdate() + "
employee deleted.");
        }
    }

    static void displayAllEmployees() throws SQLException
{
        String sql = "SELECT id, name, salary FROM
employee ORDER BY id";
        try (Connection con = getConnection();
             PreparedStatement ps =
con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                System.out.println(rs.getInt("id") + " - "
+
                        rs.getString("name") + " - " +
rs.getDouble("salary"));
            }
        }
    }

    public static void main(String[] args) {
        try {
            insertEmployee(101, "Amit", 60000);
            searchEmployee(101);
            updateSalary(101, 65000);
            displayAllEmployees();
            deleteEmployee(101);
        } catch (SQLException e) {
            System.out.println("Database error: " +
e.getMessage());
        }
    }
}
```

Expected Output

Output depends on the database contents, for example:
1 employee inserted.
101 - Amit - 60000.0
1 employee updated.
101 - Amit - 65000.0
1 employee deleted.

Important Interview Points

• Example table: CREATE TABLE employee (id INT PRIMARY KEY, name VARCHAR(100), salary DECIMAL(10,2));

• The MySQL JDBC driver must be on the runtime classpath. Database URL, username, and password should normally come from configuration, not source code.

34

Question 34: Custom Annotation for User Permissions

Scenario

A web application has methods that should only be accessible to administrators. Create a custom @AdminOnly annotation and identify annotated methods using reflection.

Topics Covered

• Custom Annotations
• Reflection
• Metadata
• Runtime Annotations
• Method Class

Starter Code

```
import java.lang.annotation.*;
import java.lang.reflect.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
```

Step-by-Step Logic

Step 1: Declare @Retention(RetentionPolicy.RUNTIME) so the annotation remains available to reflection after compilation.

Step 2: Declare @Target(ElementType.METHOD) to restrict the annotation to methods.

Step 3: Annotate the protected method with @AdminOnly.

Step 4: Obtain Class<UserService> and then its declared methods.

Step 5: For each Method, call isAnnotationPresent(AdminOnly.class) and print protected method names.

Complete Java Solution

```
import java.lang.annotation.*;
import java.lang.reflect.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface AdminOnly {
}

class UserService {
    @AdminOnly
```

```
@interface AdminOnly {
}

class UserService {
    @AdminOnly
    public void deleteUser() {
        System.out.println("User deleted");
    }

    public void viewUser() {
        System.out.println("User viewed");
    }
}

public class AnnotationDemo {
    public static void main(String[] args) {
        Class<UserService> clazz =
UserService.class;
        Method[] methods =
clazz.getDeclaredMethods();

        for (Method method : methods) {
            // TODO: Check whether @AdminOnly is
present
            // TODO: Display protected method
name
        }
    }
}
```

```
    public void deleteUser() {
        System.out.println("User deleted");
    }

    public void viewUser() {
        System.out.println("User viewed");
    }
}

public class AnnotationDemo {
    public static void main(String[] args) {
        Class<UserService> clazz = UserService.class;
        Method[] methods = clazz.getDeclaredMethods();

        for (Method method : methods) {
            if
(method.isAnnotationPresent(AdminOnly.class)) {
                System.out.println("Admin-only method: " +
method.getName());
            }
        }
    }
}
```

Expected Output

Admin-only method: deleteUser

Important Interview Points

• Annotations attach metadata; reflection inspects that metadata at runtime.

• Real authorization should be enforced by a trusted security layer rather than relying only on an annotation label.

35

Question 35: Payment System Using Strategy Design Pattern

Scenario

An online shopping application allows customers to pay using Credit Card, UPI, or Net Banking. The payment mechanism should be changeable without modifying the ShoppingCart code.

Topics Covered

• Strategy Pattern
• Interfaces
• Composition
• Polymorphism
• Loose Coupling
• Software Design Principles

Starter Code

```
interface PaymentStrategy {
    void pay(double amount);
}

class CreditCardPayment implements
PaymentStrategy {
    @Override
    public void pay(double amount) {
        // TODO: Credit card processing
    }
}

class UPIPayment implements PaymentStrategy {
    @Override
    public void pay(double amount) {
        // TODO: UPI processing
    }
}

class NetBankingPayment implements
PaymentStrategy {
    @Override
    public void pay(double amount) {
        // TODO: Net banking processing
    }
}

class ShoppingCart {
    private PaymentStrategy paymentStrategy;
```

Step-by-Step Logic

Step 1: Define a PaymentStrategy interface containing pay().

Step 2: Create one strategy class per payment algorithm.

Step 3: Store a PaymentStrategy reference inside ShoppingCart. This is composition: the cart has a strategy.

Step 4: Provide setPaymentStrategy() so the strategy can change at runtime.

Step 5: checkout() delegates payment to the current strategy and does not contain payment-type if/else logic.

Step 6: Add a guard for the case where no strategy was selected.

Complete Java Solution

```
interface PaymentStrategy {
    void pay(double amount);
}

class CreditCardPayment implements PaymentStrategy {
    @Override
    public void pay(double amount) {
        System.out.println("Paid " + amount + " using
Credit Card");
    }
}

class UPIPayment implements PaymentStrategy {
    @Override
    public void pay(double amount) {
        System.out.println("Paid " + amount + " using
UPI");
    }
}

class ShoppingCart {
    private PaymentStrategy paymentStrategy;

    public void setPaymentStrategy(PaymentStrategy
strategy) {
        this.paymentStrategy = strategy;
    }

    public void checkout(double amount) {
        if (paymentStrategy == null) {
            throw new IllegalStateException("Select a
payment strategy first.");
        }
        paymentStrategy.pay(amount);
    }
}
```

```
    public void
setPaymentStrategy(PaymentStrategy strategy) {
        // TODO: Store selected strategy
    }

    public void checkout(double amount) {
        // TODO: Use selected strategy
    }
}

public class StrategyDemo {
    public static void main(String[] args) {
        ShoppingCart cart = new ShoppingCart();

        // TODO: Select Credit Card
        // TODO: Perform payment
        // TODO: Change strategy to UPI
        // TODO: Perform another payment
    }
}
```

```
public class StrategyDemo {
    public static void main(String[] args) {
        ShoppingCart cart = new ShoppingCart();

        cart.setPaymentStrategy(new CreditCardPayment());
        cart.checkout(1500);

        cart.setPaymentStrategy(new UPIPayment());
        cart.checkout(750);
    }
}
```

Expected Output

Paid 1500.0 using Credit Card
Paid 750.0 using UPI

Important Interview Points

• Strategy favors composition over a large conditional statement inside ShoppingCart.

• Adding NetBankingPayment only requires a new PaymentStrategy implementation; checkout() does not need to change.